

第二部分·AI Agent 编程

第 9 章

AI Agent 编程基础

本章配套：正文小节 / 上机 lab / 练习 quiz / 常见误区与技巧

9. AI Agent 编程基础

本页是**第二部分（AI Agent 编程）**第 9 章的章首页。基线版本：dsh 0.1.5-rc.2 (npm latest, 2026-09-10 发布)；教材核对日期 2026-09-19；维护件见 docs/build/dsh_version.yaml。

本章概览

第一部分教会你用 NumPy、SciPy、Pandas、Matplotlib、NetworkX、Statsmodels、scikit-learn 做科学计算；从本章开始，我们换一个视角：**当 AI 能自己读文件、跑命令、改代码、看报错时，科学计算的工作方式会变成什么样？**

本章以 DeepSeek Harness (dsh) 为例——一个“一切皆插件”的开源 agent harness，讲清三件事：它是什么（概念与循环）、怎么装起来（三条路径 + API Key）、怎么安全地用（权限与数据红线）。

学习目标

1. 说清代码补全、对话式编程与 Agent 编程的区别，描述一次 Agent 循环五个动作；
2. 用“五件套”（模型、工具、上下文、权限、会话）解释一个 harness 的组成；
3. 独立完成 dsh 的三条安装路径之一（npx 免安装 / 全局 CLI / 桌面版源码构建）；
4. 完成 DeepSeek API Key 的注册、实名、充值、创建与填写，并估算一次任务的花费；
5. 会用界面与斜杠命令（/plan、/compact、/goal、/model、/permission、/export 等）控制会话；
6. 说出权限三档、审批机制与数据红线，知道什么数据不能交给云端模型。

先修要求与运行环境

- 先修：第 0 章（终端、Python 环境）与第 1 章 NumPy（本部分案例会用 axis、广播、形状判据）；
- 运行环境：Node.js ^22.19.0 || >= 24.0.0 (npm 与 npx 随它一起安装)、一个 DeepSeek 开放平台账号与 API Key；
- 详细安装步骤见本章 02 节；无 Key 时也能完成理论、审查类练习与离线替代路径。

本章结构

小节	内容	状态
01 AI 编程与 Agent 基础	三代工具、harness 五件套、循环、上下文、人机分工、失败模式	已交付
02 安装与配置	Node.js 与 npm、三条安装路径、API Key 申请与付费、价格快照、故障排查	已交付
03 界面与斜杠命令	Web 界面导览、斜杠命令速查与示例（本地执行 vs 模型对话）	编写中
04 权限、安全与数据红线	权限三档、审批实践、数据红线与合规	编写中

资源导航

资源	位置	说明
理论篇	01-AI 编程与 Agent 基础.md	概念、循环、上下文、权限、失败模式

资源	位置	说明
安装与配置	02-安装与配置.md	三条路径、API Key 与付费、成本估算、故障排查
参考资料	references.md	官方文档、平台入口、通识材料
教学说明	teaching.md	教师用：课时、演示脚本、常见坑
练习	../11-agent-cases/exercises/README.md	全部分练习统一入口（quiz + assignment）
上机	../11-agent-cases/lab/README.md	全部分上机入口（离线可跑 + 联网任务）

版本与时效

本章内容与 dsh 版本强相关，因此有三层防护：

- 1. 版本单一事实源：docs/build/dsh_version.yaml（基线版本、核对日期、上游链接、需复核文件清单）；
- 2. 时效标记：正文中易变段落以“时效内容（基线 vX / 日期）”开头，便于批量复核；
- 3. 双周巡检：docs/build/check_dsh_release.py 与 .github/workflows/dsh-release-watch.yml 在官方发新版时提醒，流程见 docs/build/dsh_更新 SOP.md。

常见问题

问题	处理
没有 API Key 能学吗？	能：本章理论、第 10 章方法、第 11 章全部审查与离线练习都不需要 Key
一定要用 DeepSeek 吗？	教材以 DeepSeek Harness + DeepSeek 模型为主线；dsh 也支持自定义兼容端点
会不会很贵？	练习任务用 deepseek-flash，一次通常几分钱；估算方法见 02 节

01 AI 编程与 Agent 基础

本节回答三个问题：Agent 编程与“AI 帮我写代码”到底差在哪？一个 Agent 内部做了什么？哪些事必须自己管？阅读约 25 分钟；配套图 4 张，可直接运行的代码 1 段。

本节目标

1. 说清**代码补全**、**对话式编程**、**Agent 编程**三者的差别，并解释“闭环”为什么是关键。
2. 用**五件套**描述一个 agent harness（智能体框架）：模型、工具、上下文、权限、会话。
3. 复述一次 **Agent 循环** 的五个动作，并能把一个小任务拆成循环。
4. 解释**上下文窗口与压缩**，知道怎么“省上下文”。
5. 说出**权限三档**与审批机制，记住数据红线在哪里。
6. 识别**四种典型失败模式**，并给出至少一条对策。

先修

- 完成第 0 章的环境安装：会用终端（PowerShell / Bash）、会运行 python 脚本；
- 学完第 1 章 NumPy：本节的演示案例用到 axis 与广播；
- 不需要任何机器学习或大模型背景。

术语速查（中英对照）

后续章节会反复用到这些词，先混个脸熟即可：

术语	英文	一句话解释
智能体	Agent	能自己调用工具、执行多步任务来完成目标的 AI 程序
智能体框架	Agent harness	把模型、工具、上下文、权限、会话组装起来跑循环的软件
工具调用	Tool call	模型请求执行一个动作（读文件、跑命令、搜索）并拿到结果
上下文	Context	模型这一次能看到的全部内容：指令、历史、文件片段、工具输出
压缩	Compaction	上下文快满时，把过程压缩成“目标 + 结论 + 证据”
权限	Permission	决定 Agent 能做什么、哪些动作需要你点头
沙箱	Sandbox	限制 Agent 只能访问指定范围的机制
技能	Skill	预先写好的领域工作流，按需加载进上下文
子代理	Subagent	由主代理派生、专门做一件事的独立 Agent
计划模式	Plan mode	先产出计划、经你确认后再动手的工作方式
会话	Session	一次持续的对话上下文，可保存、继续、回看
工作区	Workspace	Agent 默认能读写的项目目录



Figure 1: 三代编程助手对比

正文

1. 三代编程助手：从“猜下一行”到“自己动手”

	代码补全	对话式编程	Agent 编程
你给它	正在写的这一行	报错、代码片段、问题	目标 + 约束 + 验收标准
它给你	猜出的下一行	代码与解释	读文件、跑命令、改代码、再验证的 完整过程
你负责	想清楚、跑通、验证	复制粘贴、串起来、验证	定义问题、定验收、做终审
出错时	你改	你改	它自己看报错再改（直到你设的验收标准满足）

三者的分水岭是**闭环**：前两代的输出是“文本”，你拿到文本后还要自己去运行、观察、判断、再回来描述问题；Agent 的输出是“完成的动作”——它有眼睛（读文件、看报错）和手（改文件、跑命令），能在这个循环里自我修正。

一句话记住：**补全帮你写一行，对话帮你写一段，Agent 帮你把事情做完——但“做对”的定义仍然由你给出。**

2. harness：模型之外的四件事

很多人以为“用了同一个模型，效果应该差不多”。实际上，同一个模型在不同 harness 里的表现可以差很远，因为决定体验的是另外四件事：

组成	作用	少了它会怎样
模型 LLM	推理与决策	没有大脑，什么也做不了
工具 Tools	读写文件、跑命令、搜索	只能说不能做，等于聊天机器人
上下文 Context	把项目指令、历史、文件内容喂给模型	它不知道你的项目规范，反复问同样的问题
权限 Permission	决定能不能写、要不要审批	要么什么都不敢做，要么风险失控

harness（智能体框架）由五件事组成

模型只是其中一件；差距往往在其余四件



Figure 2: harness 的组成

- 判：按你给的验收标准（形状必须是 (4,)、数值与手算一致）给出结论与修复。
- 关键认识：**循环的终止条件由你定义**。没有验收标准，Agent 就会以“看起来能跑”作为终点。

4. 上下文：Agent 的短期记忆

上下文窗口再大也会满。一次长任务里，系统提示、对话历史、读入的文件、工具输出会迅速堆积；快满时 harness 会做**压缩 (compaction)**，把过程浓缩成目标、结论与关键证据。

三条实用建议：

- 给最小可复现**：一段 20 行的复现脚本，比 2000 行项目源码更有用。
- 让它自己读文件**：说“读 src/clean.py 里 load_data 函数”比把代码整段贴进对话更省上下文，也更准确。
- 长任务主动收口**：阶段结束时让它总结“已确认的结论 + 未解决的问题 + 下一步”，这既是压缩，也是你检查方向的时机。

案例卡 1：一个“看起来都对”的错误——axis 用反了 目标：认识“静默数值错误”——代码能跑、数字看着合理，但结论是错的。

代码

```
import numpy as np
np.set_printoptions(precision=2, suppress=True)

scores = np.array([[82, 90, 78],
                   [60, 65, 70],
                   [95, 88, 92],
                   [70, 72, 68]], dtype=float)
```

一次 Agent 循环的解剖

未达标就继续下一轮，达标才交付；必要时请人介入

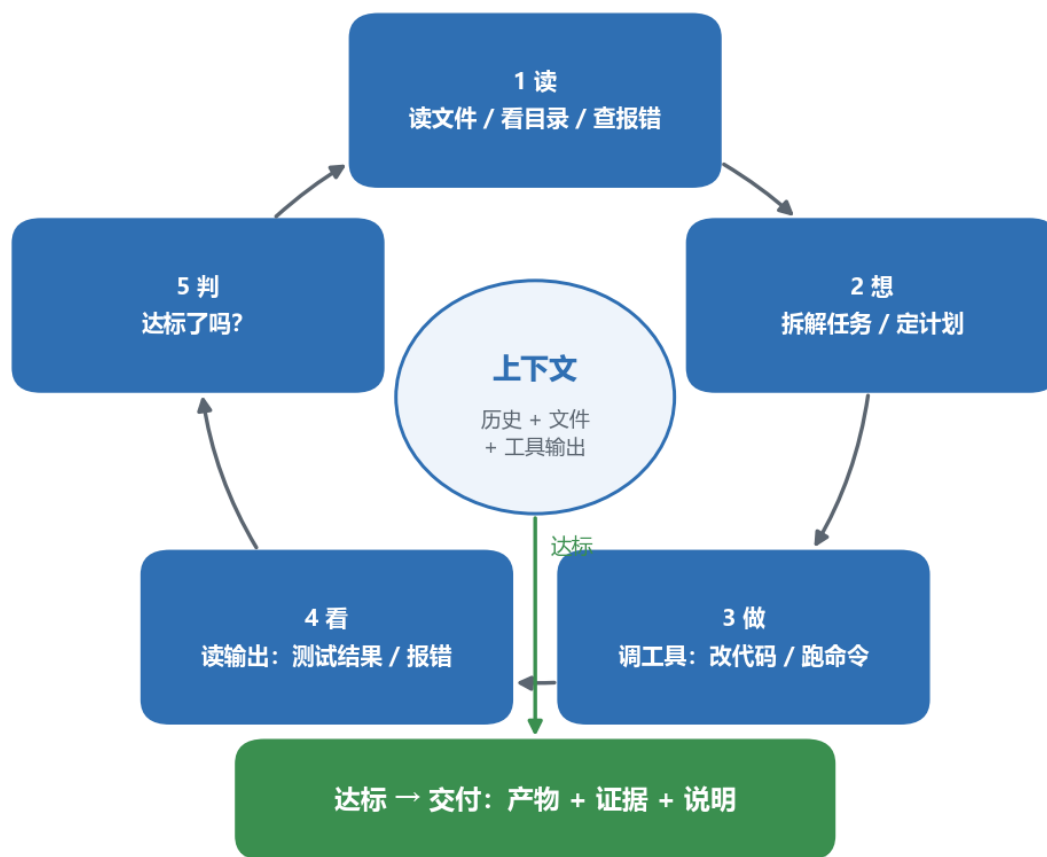


Figure 3: 一次 Agent 循环



常见错误

- 1. 只把结果打印出来看一眼“像不像”，不用形状与守恒量检查；
- 2. 把 axis=0 记成“对每一行”（其实是沿行方向压缩，得到每列的值）；
- 3. 让 Agent 修 bug 时只说“结果不对”，不说**期望的形状与判据**，它只能猜。

拓展

- 把 scores 换成 (3, 4, 5) 的三维数组，先说出 mean(axis=1) 的形状，再运行验证；
- 让 Agent 写一个 3 行的断言函数，把“形状必须是 (4,)”固化成测试；
- 思考：如果这份数据是某次实验的唯一结果，这个错误会怎样影响结论？

5. 人机分工：什么交给它，什么必须自己管

任务类型	建议	原因
样板代码、格式转换、批量重命名	交给 Agent	规则明确、易验证
写测试、补边界条件、重构	交给 Agent（你审 diff）	有客观判据
查文档、读陌生代码库、写注释	交给 Agent	信息检索型工作
问题定义与建模	人主导	决定“做什么”，Agent 无法替你决定
验收标准与数据判据	人主导	决定“什么叫做对”
结论与解释	人主导 （Agent 只能起草）	涉及因果、局限与责任

三条铁律：**问题定义归人、验收标准归人、结论归人**。Agent 可以在这三件事上给建议，但不能替你签字。

6. 权限与安全：让 Agent 在笼子里干活

主流 harness 都提供分档权限，常见三档：

档位	含义	适用场景
read-only	只能读，不能改	第一次接触陌生项目、只想让它解释代码
workspace-write	只能改工作区内的文件	日常开发、课程作业（ 推荐默认 ）
danger-full-access	全盘可写、命令直通	受控环境下的高级用法，慎用

配套机制是**审批**：涉及写文件、跑命令、装依赖时，界面会先问你；你的每一次允许或拒绝都会写进会话记录，可回看、可追责。

数据红线（请直接记住）：

- 不要把隐私数据（身份证、手机号、成绩单、医疗记录）贴进对话；
- 不要把涉密或未发表的数据交给云端模型；
- 不要把别人的 API Key、密码、令牌放进提示词或代码；
- 处理真实数据前，先想清楚“这份数据能不能离开这台机器”。

7. 四种典型失败模式与对策



Figure 5: 权限档位与审批

失败模式	长什么样	对策
幻觉 API 静默数值错误	编出不存在的函数、参数、配置项 能跑、数字合理，其实方向或单位错了	要求它运行验证；自己查一眼官方文档 事先给判据：形状、量纲、守恒量、解析解对照
过度自信的结论	把相关说成因果、不给不确定性	要求给证据、给区间、给反例，结论自己写
改坏别处	顺手重构、动了不该动的文件	小步提交、看清 diff、跑测试再接受

这四种失败都不是”模型不够聪明”，而是**缺验证**。第 03 节会给出一个固定的验收 workflow。

8. 上手体验（5 分钟）

如果你已经按第 02 节装好了 DeepSeek Harness，现在就可以试一次。任务：让它解释本节的 axis 案例。
我在做课程第 1 章的练习题，这段代码想算” 每个学生的平均分”，但我不确定对不对：

```
scores = np.array([[82, 90, 78], [60, 65, 70], [95, 88, 92], [70, 72, 68]], dtype=float)
by_exam = scores.mean(axis=0)
```

- 请你：
- 1) 指出这段代码算出来的到底是什么（给出形状）；
 - 2) 写出正确写法，并给出一个最小复现脚本；
 - 3) 加一条断言，保证结果形状符合” 每个学生” 的意图。
- 不要修改我工作区里的其他文件。

观察三个点：

1. 它有没有**先读文件、先运行**，还是直接给答案；
2. 它给出的断言是不是可执行、可验证；
3. 它有没有解释“为什么 axis=0 得到的是每列”，而不是只给结论。

本节小结

1. 三代工具的分水岭是**闭环**：能否自己执行、观察、修正。
2. harness = 模型 + 工具 + 上下文 + 权限 + 会话，后四件决定体验。
3. 循环五步：读、想、做、看、判；**终止条件由你的验收标准定义**。
4. 上下文是最稀缺资源：给最小复现、让它自己读文件、长任务主动收口。
5. 权限三档 + 审批 + 数据红线；问题定义、验收标准、结论永远归人。

思考题

1. 同一段代码，让“对话式助手”和“Agent”分别修一个 bug，你能观察到哪些行为差异？
2. 为什么“代码能跑通”不能作为验收标准？请举一个本课程里的例子。
3. 如果上下文快满了，你会保留哪三类信息？
4. 什么情况下应该主动把权限降到 read-only？
5. 同学说“Agent 写的报告结论和我们实验一致，所以没问题”。请指出这句话的逻辑漏洞。

练习与延伸阅读

- 练习：第 11 章 exercises/ 第 1-4 题（概念题）；
- 延伸：DeepSeek Harness 官方仓库与文档（见 references.md）；
- 下一节：02-安装与配置.md（三条安装路径 + API Key 申请与付费）。

02 安装与配置

时效内容（基线 v0.1.5-rc.2 / 2026-09-19）：本节涉及版本号、命令、界面与价格，是全书最容易过期的部分。遇到不一致时以官方文档为准；更新流程见 docs/build/dsh_更新 SOP.md。

本节目标

- 1. 按自己的场景选对安装路径：**免安装 Web 版 / 全局 CLI 版 / 桌面版**。
- 2. 独立完成 **API Key** 的注册、实名、充值、创建与填入。
- 3. 配置模型并跑通**第一个任务**，知道怎么验收。
- 4. 说清 dsh 的**版本状态**，会自查版本与升级。
- 5. 会用价格表**估算一次任务的花费**。

先修

- 第 0 章的终端使用与 Python 环境；
- 会看命令行输出、会复制粘贴命令即可，不需要前端或运维经验。

0. 安装前的准备（先读 2 分钟）

0.1 dsh 的版本状态与时效

DeepSeek Harness (dsh) 目前处于官方所说的 **开发者预览 (Developer Preview)** 阶段，迭代很快：

- **教材基线：**0.1.5-rc.2 (npm 的 latest 标签，2026-09-10 发布；教材核对日期 2026-09-19)；
- **官方最新：**撰写时 GitHub Releases 上所有 tag 都标记为 prerelease，没有传统意义的“稳定版”；
- 也就是说：**npm latest 就是官方当前推荐的安装版本**，名字里的 rc 不代表“不能用于教学”。

三条自查命令（建议收藏）：

```
dsh --version           # 本机已装版本
npm view @deepseek-ai/dsh dist-tags  # 官方各渠道最新版本 (latest / next / alpha)
npm view @deepseek-ai/dsh version    # latest 渠道的具体版本号
```

官方入口（完整清单见 references.md）：

- 仓库：<https://github.com/deepseek-ai/deepseek-harness>
- 文档站：<https://deepseek-harness.github.io/deepseek-harness/>
- 版本记录：仓库 Releases 页（每个版本都有中文更新说明）

0.2 Node.js、npm、npx 是什么

dsh 本身是用 JavaScript / TypeScript 写的，所以需要有一个 JavaScript 运行时；它的安装包发布在 npm 仓库里，所以还需要一个包管理器。三者关系一句话：

Node.js 提供运行环境 → npm 负责从仓库取包、装包 → dsh 是 npm 上的一个包。

名词	全称	作用	本节里第一次用到
Node.js	——	让 JavaScript 脱离浏览器运行的运行时	运行 dsh
npm	Node Package Manager	Node 的官方包管理器：装包、升级、卸载	npm i -g @deepseek-ai/dsh

名词	全称	作用	本节里第一次用到
npx	npm exec	npm 自带的“临时执行器”：不永久安装，直接下载并运行	npx @deepseek-ai/dsh web

关键点：**安装 Node.js 时会自动装上 npm 与 npx**，不需要（也不应该）单独去装 npm。网上“先装 node 再装 npm”的教程是过时说法。

0.3 安装 Node.js（三平台 + 两种风格）

版本要求：^22.19.0 或 >= 24.0.0。推荐装 **Node 24 LTS**（当前 LTS 代号 Krypton, 24.x）；**Node 22 LTS**（Jod, 22.19 以上）也满足要求。

平台	方式	步骤 / 命令	适合谁
Windows	官网安装包（推荐）	打开 nodejs.org → 下载 LTS 版.msi → 双击安装（保持默认勾选 Add to PATH）→ 重开终端	绝大多数同学
Windows	winget	winget install OpenJS.NodeJS.LTS	已用 winget 管理软件的同学
Windows / macOS	版本管理器 fnm	winget install Schniz.fnm 或 brew install fnm, 再执行 fnm install 24 与 fnm use 24	需要多个 Node 版本共存
macOS	Homebrew	brew install node@24	已装 Homebrew
macOS / Linux	nvm	按 nvm 官方 README 提供的脚本安装，然后 nvm install --lts	服务器、多版本场景
Linux	发行版包管理器 / NodeSource	apt、dnf 等； 注意仓库里的版本可能偏旧 ，装完务必按 0.4 自查	有 root 权限的服务器

两条经验：

- 1. 装完**一定要重开终端**（环境变量 PATH 才会生效），否则会一直提示找不到命令；
- 2. 有版本管理器时优先用它——将来 dsh 要求更高版本时，一条命令就能切换。

0.4 装完自查

```
node -v          # 期望 v24.x 或 v22.19 以上
npm -v           # 能打印版本号即可
npx --version    # npx 随 npm 一起安装
npm config get registry # 默认 https://registry.npmjs.org/
```

如果 npm config get registry 显示的是公司内网地址，说明这台机器配置了私有仓库，安装 dsh 前请先确认该仓库能拉到公开包（否则改用个人电脑，或按管理员要求配置代理）。



Figure 1: 三条安装路径

0.5 装不上怎么办

现象	可能原因	处理
node 不是内部或外部命令 / command not found	没装成功，或 PATH 未生效	重开终端；仍不行则重装（Windows 安装时保持勾选 Add to PATH）
npm 报 EACCES / EPERM 权限错误	用管理员或 root 装了全局包	不要用 sudo / 管理员装全局包；用普通用户重装 Node 后再试
下载很慢或超时	网络问题	临时换镜像：npm config set registry https://registry.npmmirror.com; 或按校网要求配置代理
版本太低（例如 v20）	系统包管理器里的版本旧	用 fnm / nvm 安装 24 LTS，或从官网重装
公司电脑装不了	安装策略限制	用官网 zip 免安装版解压后手动加 PATH，或改用学校机器
卸载后旧版本还在	多个 Node 共存	先查来源：where node / which -a node，再清理

1. 三条路径怎么选

路径	适合谁	需要准备	升级方式
A 免安装 Web 版	第一次用、机房机器、不想改系统	Node.js	每次运行自动取最新（也可锁定版本）
B 全局 CLI 版	长期使用、要写脚本、要无界面跑任务	Node.js + npm	npm i -g @deepseek-ai/dsh@latest
C 桌面版	想要原生窗口与自带运行时	Node.js + pnpm + 源码构建	重新拉源码构建

共同前置只有一个：**Node.js**（npm 与 npx 随它一起安装，见 0.2 与 0.3）。官方仓库要求的版本是 ^22.19.0 或 >= 24.0.0。

```
node -v      # 例如 v24.19.0
npm -v       # 例如 11.x
```

如果 Node 版本过低，先升级：官网下载 LTS 版，或用 fnm / nvm 等版本管理器（第 0 章介绍过管理多个工具链的思路）。

提示：npm 下载慢时，可以临时用国内镜像（npm config set registry https://registry.npmirror.com）；企业网络或校园网如有代理，请按网络管理员要求配置。

2. 路径 A：免安装 Web 版（推荐先试）

不安装任何全局命令，直接用 npx 拉起浏览器界面：

```
npx @deepseek-ai/dsh web
```

会发生什么：

- 1. 首次运行会下载依赖包（视网络情况可能需要几分钟），请耐心等待；
- 2. 服务默认在 http://127.0.0.1:3080 启动，本机运行时会自动打开浏览器；
- 3. **启动命令时所在的目录，就是默认工作区**，建议在一个专门的项目目录里执行；
- 4. 关闭：回到终端按 Ctrl+C。

常用参数：

```
npx @deepseek-ai/dsh web --port 8080      # 换端口（被占用时很有用）
npx @deepseek-ai/dsh web --no-open         # 只启动服务，不开浏览器
npx @deepseek-ai/dsh web --help           # 查看当前版本支持的全部参数
```

说明：如果通过 SSH 在服务器上启动，终端只会打印地址，需要你本地做端口转发（进阶用法，本节不展开）。

3. 路径 B：全局 CLI 版（长期使用）

装一次，长期用：

```
npm i -g @deepseek-ai/dsh      # 全局安装
dsh --version                  # 确认版本
dsh web                        # 等价于 npx 的浏览器界面
```

本地版真正多出来的能力是**无界面（headless）任务**——让 Agent 跑完一件事直接打印结果，适合写进脚本：

```
dsh --profile headless "统计当前目录下所有 Python 文件的行数，输出一个表格"
```

其他自查入口：

```
dsh --help           # 启动器自身的参数
dsh web --help       # Web 应用的参数
dsh --dump-config    # 查看合并后的配置树（不启动）
```

首次运行会在用户目录下创建配置与数据目录（Windows 为 C:/Users/你的用户名/.dsh，macOS 与 Linux 为 ~/.dsh），里面包括设置、会话记录与凭据文件。**凭据文件不要提交到 git，也不要截图外发。**

升级与卸载：

```
npm i -g @deepseek-ai/dsh@latest    # 升级到最新
npm rm -g @deepseek-ai/dsh          # 卸载
```

对比项	npx 免安装	全局 CLI	headless
是否改系统	否	是（全局包）	是
界面	浏览器	浏览器	无（打印结果）
适合	试用、机房	日常使用	批处理、自动化
升级	每次运行自动解析	手动升级	跟随全局

4. 路径 C：桌面版（进阶，官方暂无预编译安装包）

先说结论：**官方目前没有发布桌面端安装包**，桌面端需要从源码构建，工程量偏大。课堂上了解它的定位与取舍即可，不要求真的构建。

桌面端是什么：它是完整 Web 应用外面的一层 Electron 壳——同一套界面与能力，多了原生窗口、菜单与系统通知，并**自带 Python / Node.js / pnpm 运行时**（对不想自己配环境的用户友好）。它默认使用 19387 端口，与 Web 版的 3080 分开。

源码构建（需要 Node 与 pnpm，命令以仓库当前版本为准）：

```
git clone https://github.com/deepseek-ai/deepseek-harness.git
cd deepseek-harness
pnpm install
pnpm run build
pnpm run package:desktop:win:x64          # Windows
# macOS Apple 芯片: pnpm run package:desktop:mac:arm64
# macOS Intel: pnpm run package:desktop:mac:x64
```

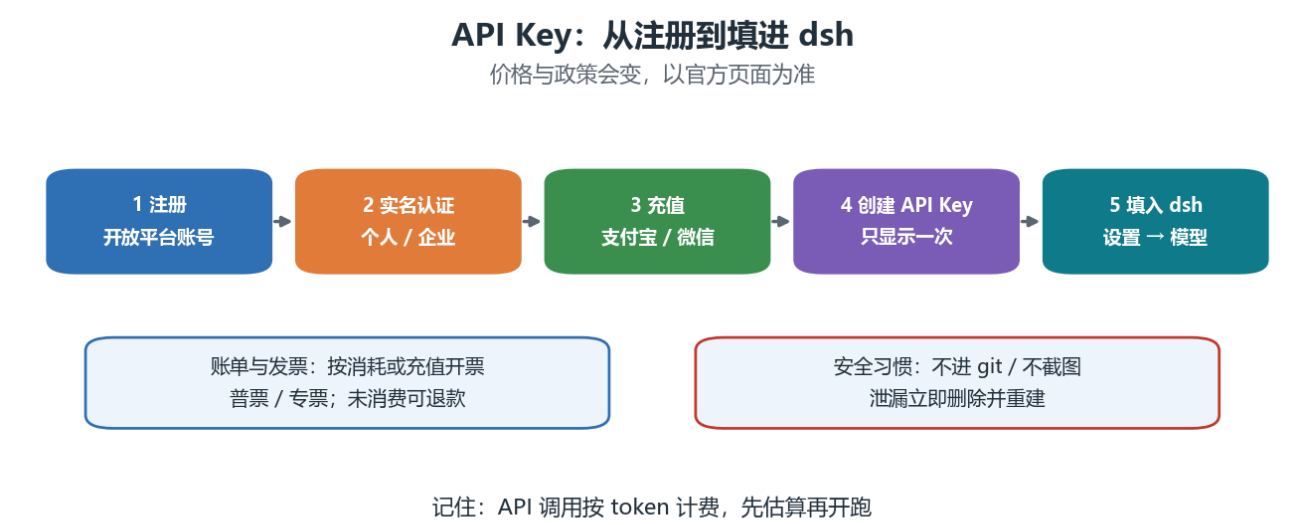
只想先看看界面，可以用开发模式启动：pnpm run dev:desktop。

社区项目：有第三方开发者为 Windows 打包了桌面版分发（例如 GitHub 上的 DeepSeek-Harness-Desktop）。它**不是官方发布**，无法审计构建过程，安装前请自行评估供应链风险；课程不把它作为推荐路径。

5. 配置模型与 API Key（重点）

5.1 为什么需要 API Key

dsh 本身是免费开源软件，但它在运行时需要调用大模型，而模型调用按用量计费。因此你需要一个 DeepSeek 开放平台账号与 API Key——可以理解为“给 Agent 用的加油卡”。



高峰时段：北京时间周一至周五 9:00–12:00、14:00–18:00；其余时间为空闲时段（半价）。价格与政策会调整，**以官方定价页为准**。课程建议：练习与作业用 deepseek-flash；把重跑安排在空闲时段，能省一半。

5.4 估算一次任务花多少钱

下面这个脚本用一组“典型参数”估算费用：20K 未命中输入 + 60K 命中输入 + 15K 输出。

```
price = {"flash": {"in_miss": 1.0, "in_hit": 0.02, "out": 4.0},
        "v4-pro": {"in_miss": 4.5, "in_hit": 0.15, "out": 13.5}}

def cost(model, in_miss_k, in_hit_k, out_k, peak=False):
    p = price[model]
    factor = 2.0 if peak else 1.0
    return factor * (p["in_miss"] * in_miss_k + p["in_hit"] * in_hit_k + p["out"] * out_k)
    / 1000.0

print("flash off-peak:", round(cost("flash", 20, 60, 15), 4), "CNY")
print("flash peak      :", round(cost("flash", 20, 60, 15, peak=True), 4), "CNY")
print("v4-pro off-peak:", round(cost("v4-pro", 20, 60, 15), 4), "CNY")

flash off-peak: 0.0812 CNY
flash peak      : 0.1624 CNY
v4-pro off-peak: 0.3015 CNY
```

结论：一次“读代码 + 改代码 + 跑测试 + 写说明”的中等任务，用 deepseek-flash 大约几分钱到几角钱；换成 deepseek-v4-pro 会贵 3 到 4 倍。

6. 第一个任务（5 分钟验收）

- 1. 在浏览器界面点「选择工作区」，添加你的课程项目目录；
- 2. 新建会话，把下面这段提示词发给它：

请阅读当前工作区，回答三个问题：

- 1) 这个项目是做什么的？列出主要目录及用途；
 - 2) 有哪些 Python 脚本？各自的作用是什么？
 - 3) 如果我要新增一个数据分析脚本，应该放在哪里、遵循什么命名习惯？
- 只读不改：不要修改任何文件。

- 3. 观察它调用了哪些工具（读文件、列目录、搜索），以及它给出的结论有没有**证据**（引用了具体文件与行）。

验收清单：

- 它确实读了文件，而不是凭空描述；
- 它引用的文件名与目录结构能在你的项目里对上；
- 整个过程没有修改任何文件（因为你在提示词里要求了只读）。

7. 常见故障排查

现象	可能原因	处理
npm: command not found	没装 Node.js	安装 Node LTS 后重开终端
npx 卡住不动	网络慢或代理	换网络、配置镜像或代理；耐心等待首次下载

现象	可能原因	处理
端口 3080 被占用	已有实例在跑	用-port 8080 换端口，或先关闭旧实例
浏览器打不开页面	服务没起来或防火墙拦截	看终端是否打印了地址、有没有报错
输入框灰色不可用	没有选择工作区	先「选择工作区」并选中
模型报鉴权错误	Key 填错或已被删除	重新在「设置 → 模型」粘贴 Key
dsh 全局命令找不到	全局 bin 目录不在 PATH	重开终端；或改回用 npx
PowerShell 执行策略报错	脚本被策略拦截	改用 npx 直接运行，或按学校要求调整策略
任务跑很久没结果	任务过大或网络慢	缩小任务范围，先给最小复现

8. 升级与版本自查

```
npm view @deepseek-ai/dsh dist-tags      # 看官方最新
dsh --version                             # 看本机版本
npm i -g @deepseek-ai/dsh@latest          # 升级
```

升级前建议读一眼 Releases 页的更新说明（官方每个版本都有中文条目），重点关注三类变化：**命令与参数**、**界面位置**、**模型名称**。这三类正是本节内容的来源。

本部分的维护机制：docs/build/dsh_version.yaml 记录基线版本，docs/build/check_dsh_release.py 定期比对官方版本，双周自动提醒（见 docs/build/dsh_更新 SOP.md）。

本节小结

- 1. 三条路径：npx 免安装（先试）、全局 CLI（长期与脚本化）、桌面版（源码构建，进阶）。
- 2. 共同前置是 Node.js（^22.19.0 或 >= 24.0.0）。
- 3. API Key 五步：注册、实名、充值、创建、填入「设置 → 模型」。
- 4. 计费按 token、分高峰与空闲；练习用 deepseek-flash，重跑放空闲时段。
- 5. Key 与凭据文件是资产：不进 git、不截图、泄漏立即删除重建。

思考题

- 1. 同学说“我用 npx 跑起来了，所以不用装 Node”——这句话哪里不对？
- 2. 为什么教材不推荐学生用第三方打包的桌面版？请列出两条风险。
- 3. 同样的任务，在高峰时段与空闲时段跑，费用差多少？请用 5.4 的脚本算一个例子。
- 4. 如果 API Key 不小心提交到了 git 仓库，你应该按什么顺序处理？
- 5. 你想让 Agent 每天晚上自动跑一次数据检查，应该选哪条路径、用什么入口？

练习与延伸阅读

- 练习：exercises/quiz.ipynb 第 5–6 题；
- 上机：lab/lab.ipynb Part A（环境自检，可无网络完成）；
- 参考：官方 README、Web UI 指南、模型配置指南（见 references.md）。

03 界面与斜杠命令

本节解决“装好之后怎么用”：Web 界面在哪里、斜杠命令怎么用、哪些操作**不经过模型**。基线版本：dsh 0.1.5-rc.2（2026-09-10）；命令清单与菜单分组随版本变化，权威说明见官方文档站。

本节目标

- 1. 认识 Web 界面的主要区域：工作区、会话、设置（模型）、权限指示；
- 2. 理解斜杠命令与普通对话的**本质区别**：命令在本地执行，不会变成模型消息；
- 3. 掌握 8 条常用命令的语法与使用时机；
- 4. 知道哪些入口**没有**命令面（headless、SDK、ACP）。

先修

- 02 节：已经装好 dsh，并完成模型与 API Key 配置。

3.1 界面导览（Web 版）

区域	位置	作用
工作区选择	首屏 / 左侧	Agent 能读写的目录边界； 没选工作区时输入框不可用
会话区	中间	对话、工具调用与结果的流水账（这就是“证据链”）
输入框	底部	输入任务；点 + 按钮或输入斜杠打开同一个菜单
设置 → 模型	右上	选择模型、填 API Key、切换推理等级
权限指示	会话附近	当前权限档位；触发写文件或命令时会弹出审批

输入框的 + 菜单与斜杠菜单是同一个（各版本措辞可能不同），分两段：

- **Add**：File（把工作区文件加入上下文）、Goal、Plan、Feedback；
- **Commands**：Compact、Permission、Model、Export。

3.2 斜杠命令是什么：不经过模型的控制面板

普通对话会作为消息进入模型上下文；**斜杠命令在本地直接执行**，命令本身与执行结果默认不会变成模型消息（因此也不消耗 token）。

命令行的语法固定：斜杠开头 + 小写名称（字母、数字、下划线、连字符）+ 空格 + 参数。名称后面的全部内容都是该命令自己的参数（各命令语法不同）。

重要限制：**命令面只在交互式界面（Web / CLI）存在**。用 headless 模式跑脚本、或通过 SDK / ACP 集成时没有这套命令，需要在提示词里用自然语言表达需求。

3.3 速查表（基线 v0.1.5-rc.2）

命令	语法	作用	什么时候用
/compact	无参数	压缩较早的会话历史	会话很长、开始变慢或答非所问时
/plan	/plan [off message], 支持附件	进入或退出计划模式	多文件改动、动数据之前
/goal	/goal [目标 clear edit 目标 pause resume]	设定或查看长任务目标	跨多轮的长任务（论文复现、项目开发）
/feedback	/feedback 文本	记录对本次会话的反馈	发现工具不好用、想留改进线索
/export	无参数（仅 Web）	把会话日志打包成 ZIP 下载	交作业 、留档、复盘
/model	打开弹窗	切换模型与推理等级	想省钱（换 flash）或想更强推理（换 pro）
/permission	打开弹窗	切换权限档位	先只读审视，再放开写
/file	打开文件选择	把工作区文件加入上下文	想让 Agent 只看某个文件时

时效内容：命令名与菜单分组由插件提供，可能随版本增删。请用斜杠菜单查看当前版本的实际清单；升级后复核本表。

3.4 三个真实用法示例

示例 1：先计划、再立目标（大任务开场）

/plan 帮我用 sklearn 的 wine 数据做一个分类小项目，先给我计划，不要动手（看完计划、确认没问题后）
/goal 完成 wine 分类小项目：脚本 + 5 折交叉验证 + 报告
要点：/plan 只负责先看计划，/goal 负责记住目标，两者都不需要额外对话轮次去解释。

示例 2：会话太长，先压缩再继续

/compact
（然后继续提要求）请把刚才的 verify.py 补上数据泄漏检查
要点：压缩会丢失部分细节，**重要结论要写进工作区文件**（如 output/review.md），不要只留在对话里。

示例 3：换模型省钱 + 收紧权限 + 导出交作业

/model # 选择 deepseek-flash（练习阶段）
/permission # 切到 workspace-write（或更小的 read-only）
/export # 会话日志打包下载，作为作业附件
要点：/export 产出的 ZIP 是“AI 使用声明”的最强证据；把它和脚本、报告一起提交。

3.5 两个最容易踩的坑

- 1. **以为命令内容会进入模型上下文**：不会。想让 Agent 知道你的目标，要用自然语言再说一遍，或写进 AGENTS.md；
- 2. **把命令当成万能开关**：命令只改会话层的行为（上下文、权限、模型、目标），不改变你的验收标准——判据仍然由你写。

3.6 五分钟练习

- 1. 在工作区里新建一个空目录，用 /plan 让 Agent 给出“读取一个 CSV 并输出摘要”的计划；

2. 确认计划后，用 `/permission` 把权限调到 `read-only`，再让它执行——观察审批与拒绝的提示；
3. 用 `/compact` 压缩会话，检查它是否还记得你的目标（如果不记得，说明目标该写进文件）；
4. 用 `/export` 下载会话日志，打开看看里面记录了哪些工具调用。

本节小结

- 斜杠命令是本地控制面：不进模型上下文、不消耗 token；
- 8 条常用命令覆盖“上下文、计划、目标、模型、权限、导出”六件事；
- 命令面只在交互式界面存在，脚本化（headless）时要用自然语言；
- `/export` 让“提交会话记录”变成一条命令，直接服务于课程作业要求。

思考题

1. 为什么“命令不进入模型上下文”既是优点也可能是坑？请举一个具体场景。
2. 会话压缩后，哪三类信息最值得提前写进工作区文件？
3. 如果你要用 headless 模式每天自动跑一次数据检查，斜杠命令还能用吗？该怎么表达“先给计划”？

练习与延伸阅读

- 练习：第 11 章 exercises/ 第 2 题（命令与场景匹配）；
- 延伸：官方文档站的 Web UI 指南与 CLI 参考（见 [references.md](#)）。

第 10 章

Agent workflow与验证方法

本章配套：正文小节 / 上机 lab / 练习 quiz / 常见误区与技巧

10. Agent 工作流与验证方法

本页是第二部分第 10 章的章首页。上一章回答“它是什么、怎么装”，本章回答“怎么干才靠谱”。

本章概览

Agent 编程的成败，很少取决于模型强弱，而取决于**方法与验证**：有没有把项目规范写进 AGENTS.md、复杂任务有没有先出计划、有没有把“什么叫做对”写成可执行的判据、关键结论有没有第二双眼睛。

本章把这套方法固化成可重复的操作流程，并在最后一节给出科研规范与学术诚信要求（AI 使用声明）。

学习目标

1. 会写最小可用的 AGENTS.md，把项目约定变成常驻指令；
2. 会用计划模式推进复杂任务，并用“小步执行 + 逐步验证”控制返工；
3. 会用**任务卡六件套**（目标、约束、验收标准、证据、复核、提交）描述任务；
4. 会为科学计算任务写出**可执行的验收标准**，并用断言把结论钉死；
5. 知道什么时候该派子代理、什么时候纯属浪费；
6. 说出 12 条常见误区，并按规范提交 AI 使用声明。

本章结构

小节	内容	状态
01 Agent 工作流与核心能力	AGENTS.md、计划先行、会话与上下文、权限实践、技能与工具、子代理、任务卡与提示词清单	已交付（后续拆分为 01-03）
02 任务卡与验收标准库	六件套、“验收标准写法的正反例”、面向第 1-8 章的判据库	编写中
03 子代理、技能与工具	角色分工、成本、技能沉淀	编写中
04 常见误区与科研规范	12 条误区、AI 使用声明模板、可复现三件套、学术诚信红线	编写中

资源导航

资源	位置	说明
工作流正文	01-Agent 工作流与核心能力.md	本章主线（含任务卡与提示词清单）
参考资料	references.md	官方文档与延伸阅读
教学说明	teaching.md	教师用：课时、案例拆解、评分要点
配套案例	../11-agent-cases/README.md	本章方法的落地演练

与前后章的关系

- 上一章（第 9 章）解决“是什么、怎么装、怎么安全用”；
- 本章解决“方法”；
- 下一章（第 11 章）用四个递进案例把方法跑一遍；
- 第 12 章解决“长期怎么维护”（版本、成本、协作、项目）。

01 Agent workflows与核心能力

本节把 01 节的原理变成**可重复的操作习惯**：项目怎么给指令、任务怎么开场、权限怎么放、结果怎么验收。配套图：子代理分工图（./images/subagent_split.png）；本节含一段可直接运行的 Python 代码。

本节目标

1. 会写一份最小可用的 AGENTS.md，让 Agent 不用每次重复问规范。
2. 会用“计划先行 + 小步执行”的方式推进复杂任务。
3. 掌握权限与审批的实践顺序：先只读、再放开、高风险动作单独确认。
4. 会用**任务卡六件套**把一个模糊需求写成可验收的任务。
5. 知道什么时候该派子代理、什么时候纯属浪费。

先修

- 01 节（Agent 循环、上下文、权限、失败模式）；
- 02 节（已经装好 dsh，并能跑通第一个任务）。

1.1 项目级指令：AGENTS.md

AGENTS.md 是放在项目里的“常驻说明”，Agent 每次工作都会先读它。它解决的是**重复解释**问题：与其每次都说“读 CSV 要指定 UTF-8、图存到 output/、随机种子固定 42”，不如写进文件。

最小可用模板（复制到你的项目根目录即可）：

项目说明

本项目是《Python 科学计算》课程的数据分析练习。

环境

- Python 3.12, 依赖见 environment.yml (numpy / pandas / scipy / statsmodels / scikit-learn / matplotlib)

- 运行命令统一用 python < 脚本名 >

约定

- 读 CSV 一律显式指定编码 utf-8-sig，并先打印列名
- 随机过程必须固定种子：np.random.default_rng(42)
- 输出文件放 output/，不覆盖 data/ 里的原始数据
- 中文图必须设置字体：Microsoft YaHei 或 SimHei

质量要求

- 每个脚本末尾要有 assert 自检；改动后必须运行一次并贴出输出
- 不确定的地方先问我，不要擅自扩大改动范围

三条经验：

1. **只写“会被违反”的约定**：写得越长越没人看，Agent 也一样；
2. **写清验收方式**：能让它自查的约定，比形容词有用；
3. **把 AGENTS.md 提交到 git**：它是项目的一部分，不是个人偏好。

1.2 计划先行：复杂任务先要计划

任务超过“改一个文件”的量级时，先让它进入**计划模式 (Plan mode)**，产出：要动哪些文件、分几步、每一步怎么验证、有什么风险。你确认后再动手。

任务规模	建议做法
一行改动、明确的小修	直接做，人审 diff
单文件重构、补测试	先说明要做什么，再执行
多文件、跨目录、动数据	必须计划模式 + 你确认
涉及删除、覆盖、联网安装	计划 + 逐条确认

计划模式的价值不是“礼貌”，而是把**返工成本**从“改完发现方向错”降到“看计划发现方向错”。

1.3 会话与上下文管理

- **一次会话只做一件事**：上下文里塞太多目标，它就容易抓错重点；
- 长任务定期收口，让它输出“已确认的结论 / 未解决的问题 / 下一步”——这三行就是压缩后的上下文；
- 换任务就开新会话，把上一轮结论作为开场信息带过去；
- 会话记录是**证据链**：提交作业时把关键会话导出或截图留档。

1.4 权限与审批：从小权限开始

推荐的实践顺序（写进课程作业要求）：

1. 第一次接触项目：用 read-only，只让它解释；
2. 确认它读懂了：切到 workspace-write，只在工作区内改；
3. 涉及删除、批量重命名、装依赖、联网、访问工作区外目录：**逐条确认**；
4. 任何情况下：不给它真实隐私数据，不让它接触你的 API Key 文件。

高风险动作清单（看到就该停下来确认）：删除或覆盖文件、git reset、批量替换、安装系统级软件、把数据上传到网络、修改环境变量。

1.5 技能 (skills) 与工具 (tools)

- **工具**是原子动作：读文件、写文件、跑命令、搜索；
- **技能**是把“一组动作 + 领域知识”打包好的 workflow，例如“按模板生成实验报告”“批量处理 Excel”。

判断标准：如果某套流程你会重复做三遍以上、且步骤固定，就值得写成技能（或一份模板文件 + AGENTS.md 说明）。

1.6 子代理：什么时候值得拆

场景	是否拆	原因
先摸清陌生代码库，再决定怎么改	拆（探索代理）	大量阅读会污染主上下文
三个互不依赖的模块同时改	拆（并行）	省时间
关键结论需要第二双眼睛	拆（评审代理）	专职找错，避免自我确认
改一个函数、加一行日志	不拆	派发与合并的开销比任务本身还大

记住：子代理不是“更强的模型”，而是**上下文隔离 + 角色分工**；每个子代理都会单独消耗 token。

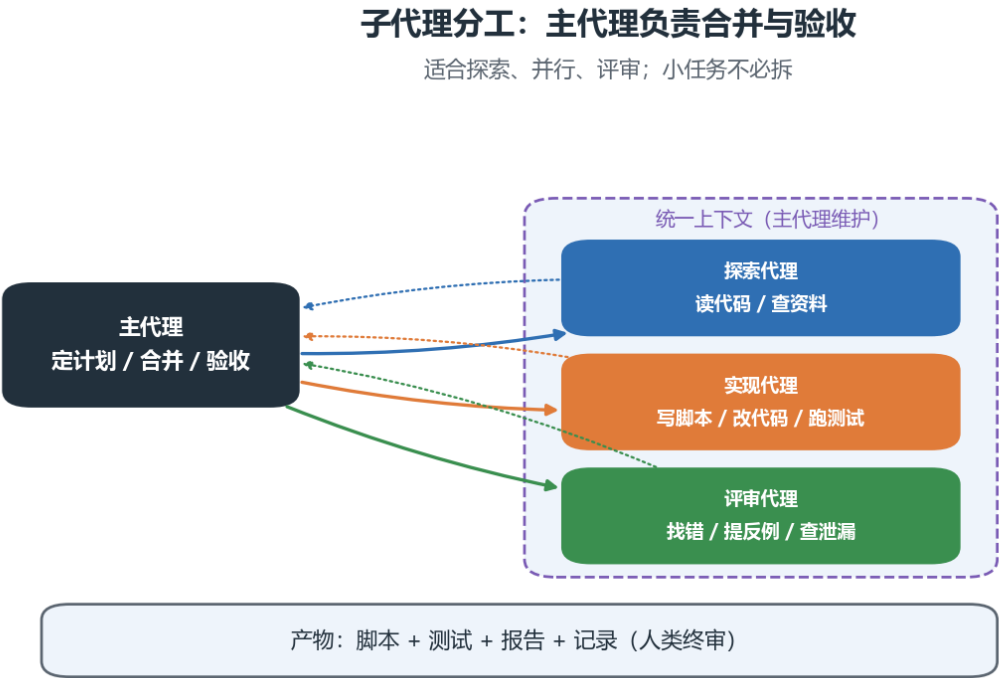


Figure 1: 子代理分工

1.7 验证驱动 workflow（本节核心）

把任何任务写成任务卡六件套：

要素	写什么	例
目标	一句话说清要达成什么	算出每个学生的平均分
约束	不许动什么、用什么环境	只新增 scores_fixed.py, 不动 data/
验收标准	可执行 的判据	长度 8、形状 (8,)、与手算一致
证据	要它给出什么证明	运行输出 + 断言结果
复核	人要检查哪几点	归一化是否逐列、排名是否带姓名
提交	交付形态	文件 + 说明 + 会话记录

验收标准写法的正反例：

差的写法	好的写法
结果要正确	平均分长度必须是 8，且与手算 [83.33, 65.0, ...] 一致
图画得好看点	图保存到 output/trend.png；x 轴为小时、y 轴为温度（摄氏度）；标题含“传感器温度”
检查有没有问题	用 assert 检查：行数守恒（清洗前后差值等于删除的重复行数）、缺失值策略已记录

为什么必须可执行：只有可执行的判据才能被反复验证。看一个最小例子——同一段代码，两种 axis，只有一种能通过断言：

```

import numpy as np
np.set_printoptions(precision=2, suppress=True)
scores = np.array([[82, 90, 78], [60, 65, 70], [95, 88, 92], [70, 72, 68]], dtype=float)

def check_student_mean(m):
    m = np.asarray(m)
    assert m.shape == (4,), " 形状必须是 (4,), 每个学生一个平均分"
    assert np.allclose(m, scores.mean(axis=1)), " 数值必须与手算一致"
    return True

for axis in (0, 1):
    try:
        check_student_mean(scores.mean(axis=axis))
        print("axis=%d -> 断言通过" % axis)
    except AssertionError as e:
        print("axis=%d -> 断言不通过: %s" % (axis, e))

axis=0 -> 断言不通过: 形状必须是 (4,), 每个学生一个平均分
axis=1 -> 断言通过

```

1.8 提示词写法清单

1. **说清目标与判据**, 不要只说“ 帮我看看”;
2. **给最小复现**: 能跑的 20 行, 胜过 2000 行项目;
3. **限定范围**: 明确“ 只改 X、不动 Y”;
4. **要求自证**: 让它跑一次并贴输出, 而不是“ 我觉得对”;
5. **要求打印中间量**: 形状、量纲、前几行、统计量;
6. **一次只做一件事**: 多个目标拆成多轮;
7. **不确定就问**: 明确告诉它“ 信息不足时先提问, 不要猜”。

1.9 一个完整示例: 把“ 帮我改改” 变成任务卡

【目标】修复 data/scores_buggy.py 中“ 每个学生平均分” 和“ 归一化” 两处错误, 产出 data/scores_fixed.py。

【约束】只新增 scores_fixed.py, 不修改 data/scores.csv 与 scores_buggy.py; 使用现有 scicomp 环境。

【验收标准】

- 1) student_mean 的形状是 (8,);
- 2) 归一化逐列进行: norm.min(axis=0) 全为 0、norm.max(axis=0) 全为 1;
- 3) 加权总分第一名是王五 (91.4 分), 且打印时带姓名。

【证据】贴出 python data/scores_fixed.py 的完整输出, 并给出三条 assert 的通过结果。

【复核】我会检查: 是否动了原始数据、归一化是否逐列、排名是否真的排序 (而不是取前三个)。

【提交】scores_fixed.py + 运行输出 + 本次会话记录。

这份任务卡就是第 11 章四个案例的统一模板。

本节小结

1. AGENTS.md 把“ 每次都要说的规范” 变成项目文件;
2. 复杂任务先计划、后执行、小步验证;
3. 权限从小往大放, 高风险动作逐条确认;
4. 任务卡六件套: 目标、约束、验收标准、证据、复核、提交;
5. 验收标准必须**可执行**, 否则等于没有。

思考题

1. 为什么“结果要正确”不算验收标准？请把它改写成两条可执行判据。
2. 什么情况下应该派评审代理？它能替代你自己的复核吗？
3. 上下文里放 AGENTS.md 有什么代价？什么时候该精简它？
4. 你打算怎么把“会话记录”变成作业的一部分？

练习与延伸阅读

- 练习：第 11 章 exercises/ 第 6–8 题（审查题与任务卡题）；
- 上机：第 11 章 lab/ Part A（审查预置产出物）；
- 下一章：第 11 章 01 案例一（读懂并修复 NumPy 代码）。

第 11 章

案例实战：从修 bug 到端到端项目

本章配套：正文小节 / 上机 lab / 练习 quiz / 常见误区与技巧

11. 案例实战：从修 bug 到端到端项目

本页是第二部分第 11 章的章首页，也是**全部分练习（exercises）与上机（lab）的统一入口**。四个案例由浅入深，每一节都包含：任务卡、给 Agent 的提示词原文、执行轨迹（真实运行记录）、可执行验收标准、人工复核点、离线替代路径。

本章概览

方法只有在真实任务里才站得住。本章用四个递进案例把第 9、10 章的概念与方法跑一遍，全部复用第 1–8 章的知识作为**验收判据**：

案例	主题	复用章节	能力层级
01 案例一	读懂并修复 NumPy 代码	第 1 章	L1 → L2
02 案例二	脏数据清洗与可视化流水线	第 4、5 章	L3
03 案例三	可复现实验与自动验证	第 3、7 章	L4
04 案例四	端到端小项目与报告	第 0、6、8 章	L5

学习目标

- 能用最小复现定位“看起来都对”的静默数值错误；
- 能把清洗策略写成可验收的判定（行数守恒、缺失与异常处理留痕）；
- 会固定种子并做重跑一致性检查，报告里区分“显著”与“解释力强”；
- 能用 Pipeline 与结构检查排除数据泄漏，并让评审代理逐条回应质疑；
- 会用 lab/ 的离线材料在没有 API Key 的环境里完成同样的审查训练。

资源导航

资源	位置	说明
练习入口	exercises/README.md	quiz（10 题）+ assignment（12 题）+ 答案
上机入口	lab/README.md	三段式 lab：离线自检 / 联网任务 / 综合项目
演示数据	lab/data/	scores.csv、scores_dirty.csv、sensor.csv、scores_buggy.py
参考产物	lab/sample_agent_output/	真实运行产物与脱敏会话记录（断网时用）
参考资料	references.md	案例涉及的数据与工具文档
教学说明	teaching.md	教师用：演示脚本、评分 rubric、断网兜底

01 案例一：读懂并修复 NumPy 代码

案例定位：L1 → L2（先只读分析，再单文件修复 + 自测）。复用第 1 章（axis、广播、形状判据）；真实运行记录见 lab/sample_agent_output/。

案例背景

课程练习脚本 `scores_buggy.py` 能正常运行、输出的数字看起来也合理，但它有两处**静默错误**：

- `scores.mean(axis=0)` 求的是“每门课的平均分”（3 个数），学员期望的是“每个学生的平均分”（8 个数）；
- `(scores - scores.min()) / (scores.max() - scores.min())` 用的是**全矩阵**的最小/最大值，把三个科目混在一起归一化，导致每列的范围不再是 0 到 1。

另外它打印的“加权总分前 3 名”其实是“前 3 个学生”，不是真正的排名。

数据：lab/data/scores.csv（8 名学生 × 平时 / 期中 / 期末，满分 100）；**待修脚本**：lab/data/scores_buggy.py。

任务卡（六件套）

要素	内容
目标	找出并修复 <code>scores_buggy.py</code> 的错误，产出 <code>scores_fixed.py</code>
约束	只新增 <code>scores_fixed.py</code> ；不修改 <code>data/scores.csv</code> 与 <code>scores_buggy.py</code>
验收标准	① 平均分形状为 (8,) 且与手算一致；② 逐列归一化后每列 min=0、max=1；③ 加权总分第一为 王五 91.4
证据	运行 <code>python scores_fixed.py</code> 的完整输出 + 三条 <code>assert</code> 通过
复核	原始数据是否被改动、归一化是否逐列、排名是否真排序
提交	脚本 + 运行输出 + 会话记录

给 Agent 的提示词（原文）

工作区里有一个成绩分析脚本 `data/scores_buggy.py` 和配套数据 `data/scores.csv`。请只读分析，不要修改任何文件：

- 1) 指出脚本里每个隐蔽错误，说明它算出来的到底是什么、为什么错；
- 2) 给出一个最小复现命令（能证明错误的形状或数值）；
- 3) 给出修正方案（关键行改成什么）；
- 4) 为修正后的结果写 3 条可执行断言（形状、逐列归一化范围、加权总分最高分是谁）。

请用简洁的条目输出。

Agent 执行轨迹（示例，基线 v0.1.5-rc.2 / 2026-09-19）

第一步：读。它先列出工作区、读脚本与 CSV，确认列名与行数（而不是直接给答案）。推理摘录：

The lab directory only has `scores_buggy.py` as a `.py` file ... the buggy script reads `data/scores.csv`.

Let me read scores.csv to understand the structure and check who 王五 is and confirm columns.

第二步：判。它给出的问题清单（要点）：

位置	它算出来的	应该是什么
student_mean = scores.mean(axis=0)	每门课的平均分，3 个数	每个学生的平均分，8 个数 (axis=1)
norm = (scores - scores.min()) / ...	全矩阵归一化：各列 min=[0, 0.25, 0.175]、max=[1, 0.975, 0.925]	逐列归一化：各列 min 全 0、max 全 1
total[:3]	前 3 个学生	按总分降序取前三名

第三步：做 + 看。它生成 scores_fixed.py、运行、把输出贴回来（真实输出）：

每个学生的平均分（共 8 个）：

```
张三    83.33
李四    65.00
王五    91.67
赵六    70.00
钱七    83.00
孙八    61.67
周九    91.33
吴十    76.67
```

归一化矩阵形状：(8, 3)
逐列归一化范围：min=[0. 0. 0.] max=[1. 1. 1.]
加权总分前 3 名：

```
第 1 名 王五    加权总分 91.40
第 2 名 周九    加权总分 90.90
第 3 名 钱七    加权总分 82.90
```

全班平均：77.83
自检通过：形状 / 逐列归一化范围 / 加权总分最高分为王五

说明：Agent 还主动指出“原脚本的 total[:3] 不是排名”，并把打印口径一起修正——这是“读 → 判 → 做 → 看”完整闭环的体现。示例输出会随版本与模型变化；**可复现的是验收标准与断言，不是这段文字。**

修复要点（关键行）

```
# 文件: data/scores_fixed.py (节选)
student_mean = scores.mean(axis=1)                                # 每个学生一个值
norm = (scores - scores.min(axis=0)) / (scores.max(axis=0) - scores.min(axis=0))  # 逐列归一化
total = scores @ weights
order = np.argsort(-total)[:3]                                    # 真正的前三名

assert student_mean.shape == (8,)
assert np.allclose(norm.min(axis=0), 0.0) and np.allclose(norm.max(axis=0), 1.0)
assert df[" 姓名"].iloc[int(np.argmax(total))] == " 王五"
```

验收标准（可执行，已运行核验）

下面这段不依赖任何数据文件，直接内联了这 8 名学生的成绩，任何机器上都能跑：

```

import numpy as np
np.set_printoptions(precision=2, suppress=True)

scores = np.array([[82, 90, 78],
                   [60, 65, 70],
                   [95, 88, 92],
                   [70, 72, 68],
                   [88, 76, 85],
                   [55, 68, 62],
                   [91, 94, 89],
                   [76, 80, 74]], dtype=float)
weights = np.array([0.2, 0.3, 0.5])
names = ["张三", "李四", "王五", "赵六", "钱七", "孙八", "周九", "吴十"]

student_mean = scores.mean(axis=1)
norm = (scores - scores.min(axis=0)) / (scores.max(axis=0) - scores.min(axis=0))
total = scores @ weights

assert student_mean.shape == (8,), "每个学生一个平均分"
assert np.allclose(norm.min(axis=0), 0.0) and np.allclose(norm.max(axis=0), 1.0), "逐列归一化"
top = int(np.argmax(total))
assert names[top] == "王五", "加权总分第一应是王五"

print("学生平均分:", np.round(student_mean, 2))
print("逐列范围 min:", norm.min(axis=0), "max:", norm.max(axis=0))
top3 = "; ".join("%s %.2f" % (names[i], total[i]) for i in np.argsort(-total)[:3])
print("加权总分前三:", top3)
print("断言全部通过")

学生平均分: [83.33 65.    91.67 70.    83.    61.67 91.33 76.67]
逐列范围 min: [0. 0. 0.] max: [1. 1. 1.]
加权总分前三: 王五 91.40; 周九 90.90; 钱七 82.90
断言全部通过

```

人工复核点

1. 有没有动原始数据: git status 里 data/scores.csv 与 scores_buggy.py 必须原样;
2. 断言是不是自我循环: 检查 assert 是否真的能卡住旧实现 (把旧写法代进去应立刻失败);
3. 排名是否真排序: 看到 argsort / sort_values 才算数;
4. 解释是否对得上: 它说"axis=0 得到每门课平均分" 必须与第 1 章的知识一致。

离线替代路径 (无网络 / 无 API Key)

直接打开 lab/sample_agent_output/:

- scores_fixed.py: 一次真实运行的修复产物;
- run_fix_session.txt: 本次会话的完整记录 (已脱敏), 含推理过程与运行输出。

练习: 找出脚本里相对原版的 4 处修改, 逐条说明"为什么必须这么改", 并用 1 条断言把其中一处钉死。

常见坑

坑	现象	避免方式
只让它“改对”，没给判据	它可能只把 <code>axis=0</code> 改成 <code>axis=1</code> ，漏掉归一化	验收标准写全三条
断言写在修复之后的代码里自证 让它顺手重构	<code>assert</code> 永远成立，抓不到回归 <code>diff</code> 变得无法审查	断言要能卡住旧实现 提示词里写明“不做无关改动”
中文输出在 Windows 控制台乱码	看到“银斤拷”类乱码	设 <code>PYTHONIOENCODING=utf-8</code> 或改在 VS Code 终端运行

拓展

1. 把这份数据扩展成“4 次测验 + 权重不同”，让 Agent 写参数化函数并补边界测试（全班缺考一人怎么办）；
2. 让评审代理（另一个子代理）去挑这份修复的毛病，看看它能不能发现“加权总分假设权重已归一化”；
3. 把 `scores_fixed.py` 改造成命令行工具：`python scores_fixed.py --top 3`，并让 Agent 补 `--help` 说明。

练习与延伸阅读

- 练习：exercises/quiz.ipynb 第 9 题（审查题：给出这段轨迹，找出仍然存在的问题）；
- 上机：lab/lab.ipynb Part B 任务 1；
- 下一节：02 案例二（数据清洗与可视化流水线）。

02 案例二：数据清洗与可视化流水线

案例定位：L3（多文件任务：清洗脚本 + 图表 + 结果表）。复用第 4 章（缺失值、重复值、分组统计）与第 5 章（图表规范、随机种子）。

案例背景

lab/data/scores_dirty.csv 是一份“真实世界风格”的成绩表：11 行、6 列，里面混着几类常见脏数据：

问题	位置	数量
学号重复	张三出现两次	1 行
学号/姓名有空白	2024009 尾部空格、王五姓名两侧空格	2 处
成绩缺失	李四的期中、期末为空	2 个
异常值	孙八期末 = 999	1 个
班级缺失	周九班级为空	1 个

任务：让 Agent 产出**可复现的清洗流水线**，并给出图表与结果表。这个案例的重点不是“洗得干净”，而是**每一步都能被验收**。

任务卡（六件套）

要素	内容
目标	产出 clean_scores.py：读原始数据 → 清洗 → 输出干净数据、图表与清洗报告
约束	不覆盖 data/scores_dirty.csv；输出统一放 output/；随机种子固定
验收标准	① 行数守恒：清洗后行数 = 原始行数 - 删除的重复行数 (= 10)；② 缺失与异常处理策略写进报告；③ 图有标题/坐标轴单位，保存为 PNG
证据	脚本运行输出 + output/ 文件清单 + 清洗报告
复核	异常值是被“修正”还是被“标记”？缺失值是删除还是填充？理由是否记录？
提交	脚本 + 输出文件 + 报告 + 会话记录

给 Agent 的提示词（原文）

请为 data/scores_dirty.csv 写一个可复现的清洗脚本 clean_scores.py，要求：

- 学号与姓名去首尾空格；班级缺失填" 未知"；
- 重复学号只保留第一条（并在报告里说明删了几行）；
- 期末成绩大于 100 视为异常，标记为缺失（不要凭空改成 100 或平均值）；
- 输出 output/scores_clean.csv、output/clean_report.md（写明每类问题的数量与处理策略）；
- 画一张图 output/score_box.png：三个科目成绩的箱线图，中文标签，标题含" 成绩分布"，保存 150 dpi；
- 脚本末尾用 assert 自检：行数守恒、姓名无空白、班级无缺失。

不要修改 data/ 下的原始文件；写完后运行并把输出贴给我。

期望产物

```
output/
├─ scores_clean.csv      # 清洗后的数据（10 行）
├─ clean_report.md       # 每类问题的数量与处理策略
└─ score_box.png        # 三科成绩箱线图（中文标签、150 dpi）
data/scores_dirty.csv    # 原始文件，保持不变
```

验收标准（可执行，已运行核验）

下面这段把“脏数据 → 清洗 → 判据”浓缩成 20 行，任何机器都能跑：

```
import numpy as np
import pandas as pd

raw = pd.DataFrame({
    " 学号": ["2024001", "2024002", "2024003 ", "2024004", "2024001"],
    " 姓名": [" 张三", " 李四", " 王五 ", " 赵六", " 张三"],
    " 班级": [" 一班", " 一班", " 一班", None, " 一班"],
    " 平时": [82, 60, 95, 70, 82],
    " 期中": [90, np.nan, 88, 72, 90],
    " 期末": [78, np.nan, 92, 999.0, 78],
})

before = len(raw)
clean = raw.copy()
clean[" 学号"] = clean[" 学号"].str.strip()
clean[" 姓名"] = clean[" 姓名"].str.strip()
clean[" 班级"] = clean[" 班级"].fillna(" 未知")
clean[" 期末"] = clean[" 期末"].mask(clean[" 期末"] > 100)
clean = clean.drop_duplicates(subset=[" 学号"]).reset_index(drop=True)
removed = before - len(clean)

print(" 原始行数:", before, " 清洗后行数:", len(clean), " 删除重复行:", removed)
print(" 行数守恒:", removed == 1)
print(" 期末缺失数（含异常值置缺）:", int(clean[" 期末"].isna().sum()))
print(" 姓名是否还有空白:", bool((clean[" 姓名"] != clean[" 姓名"].str.strip()).any()))
print(" 班级取值:", sorted(clean[" 班级"].unique().tolist()))
```

原始行数：5 清洗后行数：4 删除重复行：1

行数守恒：True

期末缺失数（含异常值置缺）：2

姓名是否还有空白：False

班级取值：['一班', '未知']

图表复核点（第 5 章标准的 Agent 版）

1. 中文是否正常显示（字体设置），有没有方框乱码；
2. 标题、x/y 轴标签、单位是否齐全；
3. 箱线图是否说明了异常点处理方式（999 被置缺后不应再出现）；
4. 图片是否保存为文件（而不是只 plt.show()）；
5. 随机过程是否固定种子（本案例无随机，但报告里应体现“可复现”意识）。

离线替代路径

无网络时：用 lab/sample_agent_output/ 里的参考实现与 data/scores_dirty.csv 自己写一遍清洗脚本，再对照验收标准逐条检查。

常见坑

坑	后果	避免方式
让 Agent“把异常值改成合理值” 没有行数守恒判据	999 被悄悄改成 100，数据被篡改 到底删了几行说不清	要求“标记为缺失”并写进报告 断言：清洗前行数 - 清洗后行数 == 删除的重复行数
图只 show 不 save 报告与脚本不一致	提交时没有图片文件 报告说“填充均值”，脚本其实删了行	验收标准里写明输出路径与 dpi 报告由脚本自动生成，不手写

拓展

- 1. 增加“按班级分组统计”与分面箱线图（第 4、5 章综合）；
- 2. 让 Agent 写第二份脚本 verify_clean.py，专门检查清洗结果（把生成与验收分离）；
- 3. 把清洗策略做成配置文件 (clean_rules.yaml)，让脚本读配置执行，便于复用。

练习与延伸阅读

- 练习：exercises/quiz.ipynb 第 9–10 题（审查题）；
- 上机：lab/lab.ipynb Part B 任务 2；
- 下一节：03 案例三（可复现实验与自动验证）。

03 案例三：可复现实验与自动验证

案例定位：L4（项目级：脚本 + 统计结论 + 自动验证 + 局限说明）。复用第 3 章（SciPy 检验）与第 7 章（Statsmodels 回归、时间序列）。

案例背景

lab/data/sensor.csv 是一段 240 小时的传感器记录（列：t 小时、温度 摄氏度、振动 无量纲）。任务假设：**温度随时间存在线性上升趋势。**

这个案例训练的不是“跑出 p 值”，而是：**固定种子、可重跑、结论有区间、局限写清楚。**

任务卡（六件套）

要素	内容
目标	产出 experiment.py + output/fit_summary.txt + output/trend.png + report.md
约束	不做数据变换前先画图；种子固定 20240919；不修改 data/sensor.csv
验收标准	① 重跑两次结果完全一致；② 报告给出斜率、p 值、置信区间与 R ² ；③ 措辞区分“相关”与“因果”；④ 报告含“局限与不确定性”一节
证据	两次运行的输出对比 + 图 + 报告
复核	p 值的含义是否说对；趋势是否被季节性掩盖；结论是否超出数据支持范围
提交	脚本 + 输出 + 报告 + 会话记录

给 Agent 的提示词（原文）

请用 data/sensor.csv 做一次可复现的统计实验 experiment.py：
1) 先画温度随时间的折线图（保存 output/trend.png，标注单位与标题）；
2) 用 OLS 拟合“温度 ~ t”，输出斜率（每小时升温）、p 值、95% 置信区间、R²；
3) 结果写入 output/fit_summary.txt；
4) 用固定种子生成同一份合成数据做一次自检，证明“同样输入 → 同样输出”（重跑一致）；
5) 写 report.md：结论 + 证据 + 局限与不确定性（例如季节性、采样频率、未测变量）。
注意：不要用“导致”“证明”这类因果措辞；脚本末尾用 assert 检查重跑一致性与显著性的判断逻辑。

验收标准（可执行，已运行核验）

这段代码用与 sensor.csv 相同的生成方式（同一种子）做数据，因此**它同时验证了“可复现”这件事本身：**

```
import numpy as np
import statsmodels.api as sm

def gen(seed):
    rng = np.random.default_rng(seed)
    t = np.arange(240)
    y = 20 + 0.02 * t + 3 * np.sin(2 * np.pi * t / 24) + rng.normal(0, 0.5, t.size)
    return t, y
```

```
t1, y1 = gen(20240919)
m1 = sm.OLS(y1, sm.add_constant(t1)).fit()
t2, y2 = gen(20240919)
m2 = sm.OLS(y2, sm.add_constant(t2)).fit()

print(" 斜率 (每小时):", round(float(m1.params[1]), 6))
print("p 值:", float(m1.pvalues[1]))
print(" 重跑一致:", bool(np.allclose(m1.params, m2.params)))
print(" 显著 (alpha=0.05):", bool(m1.pvalues[1] < 0.05))

斜率 (每小时): 0.018037
p 值: 1.469930277503604e-16
重跑一致: True
显著 (alpha=0.05): True
```

用 lab/data/sensor.csv（同样数据、只保留 3 位小数）得到的结果一致：斜率 0.018037、 $p \approx 1.47\text{e-}16$ 、 $R^2 \approx 0.25$ 。注意 R^2 ：温度只有约 25% 的波动由线性趋势解释——**显著不等于解释力强**，这正是报告里要写的“局限”。

人工复核点

- 1. **种子的作用范围**：固定种子只保证随机数一致，不保证结论正确；
- 2. **p 值的读法**：p 小说明“在零假设下出现这种数据的概率低”，不等于“效应大”；
- 3. **季节性**：24 小时周期很强，只拟合线性趋势会低估波动，报告要说明；
- 4. **措辞**：允许写“与.....一致/相关”，不允许写“证明了/导致了”；
- 5. **可复现三件套**：脚本、数据、环境（版本号）都在提交物里。

离线替代路径

无网络时：直接用本地 scicomp 环境运行教学脚本（本案例的 acceptance 代码不依赖 dsh）；再把 Agent 的报告当作“待审稿件”，逐条找出超出证据的措辞。

常见坑

坑	后果	避免方式
不固定种子	别人重跑得到不同数字	断言“两次重跑一致”
只报 p 值不报效应量	结论被夸大	同时给斜率、置信区间、 R^2
忽略季节性直接下结论	把周期当趋势	先画图再建模
用“证明”下因果结论	学术表达不严谨	报告模板里固定“局限”一节

拓展

- 1. 在回归里加入“小时”周期项 (sin/cos)，比较 R^2 的变化；
- 2. 用第 7 章的 ARIMA 对残差建模，检查是否还有自相关；
- 3. 让评审代理扮演“审稿人”，只允许它对报告提出 3 条必须要修改的质疑。

练习与延伸阅读

- 练习：exercises/quiz.ipynb 第 10 题；
- 上机：lab/lab.ipynb Part B 任务 3；
- 下一节：04 案例四（端到端小项目与报告）。

04 案例四：端到端小项目与报告

案例定位：L5（多角色协作 + 人类终审 + LaTeX 报告）。复用第 0 章（LaTeX 报告）、第 6 章（网络分析）、第 8 章（sklearn 全流程）。

案例背景

前面三个案例都是“修一处、做一版”；本案例是一次完整的小项目：给定数据集与问题，走完“探索 → 建模 → 评审 → 报告”的全流程，并且由你担任项目负责人——Agent 是执行团队，不是决策者。

两个可选题目（与期末大作业同源，任选其一）：

题目	数据	任务	主要章节
A 葡萄酒分类	sklearn 自带 load_wine	分类 + 聚类对照 + 特征重要性	第 8 章
B 交通网络分析	第 6 章案例数据 或 自备路网	中心性 / 最短路 / 社区 + 可视化	第 6 章

本案例以 A 为例，交付物包括：项目骨架、AGENTS.md、可复现脚本、评审记录、report.tex（编译成 PDF）。

任务卡（六件套）

要素	内容
目标	完成一个可复现的建模小项目，并输出 LaTeX 报告
约束	预处理必须在 Pipeline 内；报告与脚本一一对应；不得使用测试集调参
验收标准	① 5 折交叉验证最低分 ≥ 0.90 ；② 结构检查：标准化在 Pipeline 内；③ 评审代理至少提出 3 条意见且逐条回应；④ 报告含数据、方法、结果、局限、AI 使用声明
证据	脚本运行输出、交叉验证明细、评审意见表、编译成功的 PDF
复核	有没有数据泄漏、指标口径是否一致、结论是否超出数据支持
提交	项目目录 + 报告 PDF + 会话记录 + AI 使用声明

给 Agent 的提示词（原文）

【项目】用 sklearn 自带 wine 数据集做一个分类小项目，目录结构：
agentproj/{AGENTS.md, src/, output/, report/report.tex}

【要求】

- 1) 先进入计划模式，给出步骤与要产出的文件，等我确认后再动手；
- 2) src/train.py: Pipeline (标准化 + 逻辑回归), 5 折交叉验证, 输出 output/cv_scores.txt;
- 3) src/verify.py: 结构检查 (标准化必须在 Pipeline 内) + 指标断言 (5 折最低分 ≥ 0.90);
- 4) 用子代理做两件事：一个探索代理分析特征分布并给出 3 条建模建议；一个评审代理只看 src/ 与 output/, 提出不少于 3 条质疑；
- 5) 把评审意见做成 output/review.md (表格: 意见 / 是否接受 / 处理方式), 逐条回应;
- 6) 最后生成 report/report.tex (中文, 含数据、方法、结果、局限、AI 使用声明), 并给出编译命令。

【约束】不修改 sklearn 自带数据；不把测试集用于任何拟合；所有随机过程固定种子。

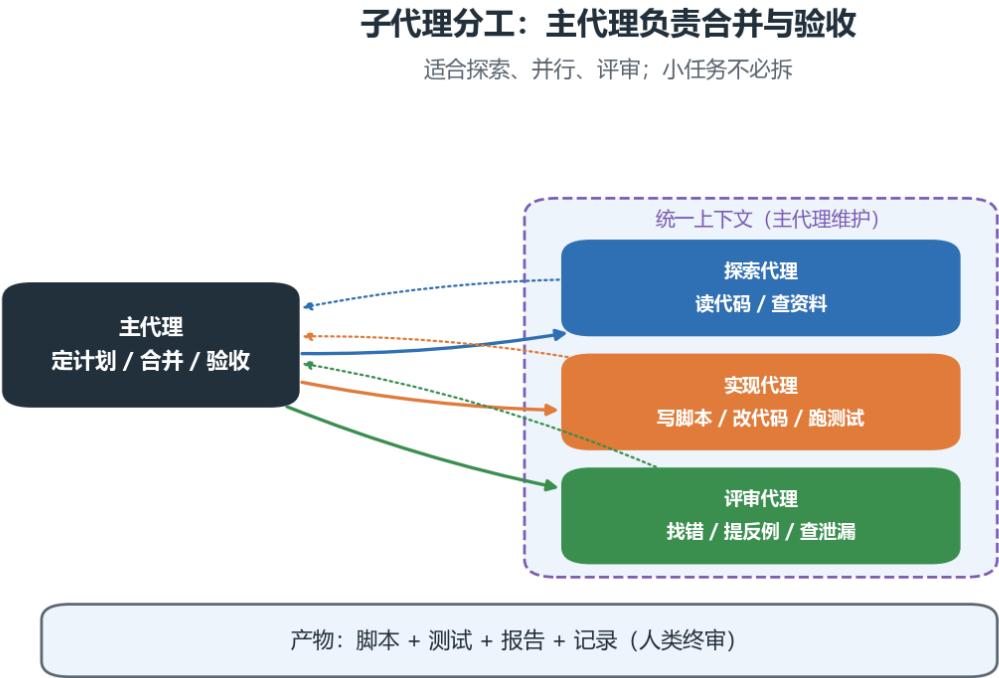


Figure 1: 子代理分工

子代理分工

角色	输入	产出	你要做的检查
探索代理	数据集与题目	特征分布、类别平衡、3 条建模建议	建议是否与数据事实一致
实现代理	任务卡 + AGENTS.md	脚本、输出文件	是否动了不该动的文件
评审代理	只给 src/ 与 output/	≥3 条质疑	质疑是否被逐条回应

验收标准（可执行，已运行核验）

```
import numpy as np
from sklearn.datasets import load_wine
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_wine(return_X_y=True)
pipe = make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000))
scores = cross_val_score(pipe, X, y, cv=5)

assert len(pipe.steps) == 2 and pipe.steps[0][0] == "standardscaler", " 标准化必须在 Pipeline 内"
assert scores.min() > 0.9, "5 折最低分不应低于 0.90"

print("5 折准确率:", np.round(scores, 4))
```

```
print(" 均值:", round(float(scores.mean()), 4))
print(" 结构检查通过: 标准化在 Pipeline 内, 无全局预处理泄漏")

5 折准确率: [0.9722 0.9722 1.      0.9714 1.      ]
均值: 0.9832
结构检查通过: 标准化在 Pipeline 内, 无全局预处理泄漏
```

注意: `scores.min() > 0.9` 这样的阈值来自**你对任务的要求**, 不是模型自带的结论; 如果你换数据集, 阈值必须重设——这正是“验收标准由人定义”。

人工复核点

- 1. **数据泄漏**: 标准化、特征选择、降维是否都在 Pipeline 内 (在划分之前做 = 泄漏);
- 2. **指标口径**: 交叉验证与测试集指标不能混用一句话描述;
- 3. **报告一致性**: 报告里的每个数字都能在 `output/` 里找到出处;
- 4. **评审闭环**: 评审意见不能只有“已修复”, 要写清怎么改、改后指标变化;
- 5. **AI 使用声明**: 写明哪些环节用了 Agent、你做了哪些修改 (模板见 08 节)。

离线替代路径

无网络时: 仍可完成本案例 (sklearn 数据自带、脚本本地跑); 把“Agent 产出”换成 `lab/sample_agent_output/` 的参考实现, 你扮演评审代理, 练习写 `review.md` 与逐条回应。

常见坑

坑	后果	避免方式
让 Agent 一把梭完成整个项目	过程不可解释, 出错难定位	先计划模式, 再分阶段执行
报告由 Agent 直接生成、数字对不上	数据与结论脱节	报告中的数字必须来自 <code>output/</code>
评审代理与实现代理“同一视角”	挑不出真问题	评审只给代码与输出, 不给你的解释
忘记 LaTeX 编译验证	提交的 tex 编不过	验收标准里加“编译成功并附 PDF”

拓展

- 1. 换成题目 B (交通网络), 比较两种任务下 Agent 的表现差异;
- 2. 引入“红队”环节: 让评审代理专门构造能骗过指标的测试样例 (例如类别不平衡);
- 3. 把整个项目接进 git: 每个阶段一次提交, 最后用 `git log` 复盘“哪些改动来自 Agent、哪些来自你”。

练习与延伸阅读

- 练习: `exercises/assignment.md` 第 11–12 题 (项目复盘 + AI 使用声明);
- 上机: `lab/lab.ipynb` Part C (综合任务);
- 延伸: 第 8 章综合案例、期末大作业项目 1 / 2。

第 12 章

工程化与持续演进

本章配套：正文小节 / 上机 lab / 练习 quiz / 常见误区与技巧

12. 工程化与持续演进

本页是第二部分第 12 章的章首页。前三章解决“会不会用”，本章解决“能不能长期用下去”。

本章概览

AI 工具迭代极快：本章写作基线是 dsh 0.1.5-rc.2，而官方在两周内就发布了多个新版本。真正可迁移的能力不是记住某个命令，而是**建立一套跟着版本走的工程习惯**：锁定版本、定期巡检、记录变更、控制成本、规范协作、按合规提交。

本章把教材自己的维护机制（版本单一事实源、巡检脚本、双周 CI、更新 SOP）作为教学案例讲清楚，并以一个综合项目收尾。

学习目标

- 1. 会为 AI 工具建立“版本单一事实源”并定期巡检（npm dist-tags + GitHub Releases）；
- 2. 会写更新 SOP：发现新版本 → 评估影响面 → 改文档 → 重跑验证 → 提交；
- 3. 会估算与控制 AI 使用成本（模型档位、token 估算、空闲时段、轮次限制）；
- 4. 会制定团队/课程的协作规范（AGENTS.md、评审、会话留档、AI 使用声明）；
- 5. 完成综合项目 Project 3：用 Agent 完成一次端到端数据分析并提交报告。

本章结构

小节	内容	状态
01 版本与时效管理	dsh_version.yaml、巡检脚本、双周 CI、更新 SOP	编写中
02 成本、协作与合规	成本治理、团队规范、安全合规、学术诚信	编写中
03 综合项目 Project 3	端到端数据分析项目（含 AI 使用声明）	编写中

资源导航

资源	位置	说明
参考资料	references.md	版本管理、CI、成本与合规资料
教学说明	teaching.md	教师用：项目布置、评分与常见问题
维护件	docs/build/dsh_version.yaml、docs/build/check_dsh_release.py、docs/build/dsh_更新 SOP.md	教材自身的版本巡检机制
期末项目	../教学资源/期末大作业/	Project 1 / 2 / 3 说明与模板